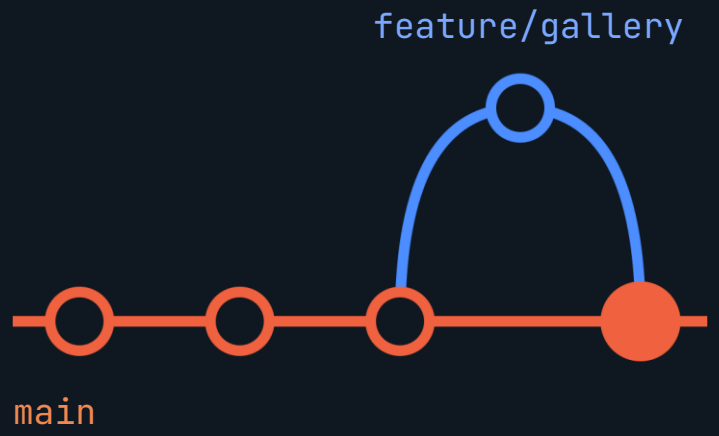
Introduction to GITHUB

Track every change, publish to GitHub, put a website live, and work as a team with branches.



[Course code] · Dr. Malek Alsmadi

THE BIG IDEA

Git and GitHub are not the same thing

Git

- Software that runs **on your computer**
- Takes snapshots (**commits**) of your project
- Works offline: a powerful undo for every file
- Shows what changed, when, and who changed it

GitHub

- A **website** that hosts Git projects online
- Backs up your code in the cloud
- Share a link so others can download it
- Lets a team contribute to the same project

BEFORE YOU START

Install these four things

1

Git

The version-control software. Accept the default install options.

`git-scm.com`

2

VS Code

Our editor. Its Source Control panel gives Git buttons instead of commands.

`code.visualstudio.com`

3

GitHub account

Free. Use an email you will keep; Use you College email.

`github.com`

Restart VS Code after installing Git so it can find it.

THE LOOP YOU WILL REPEAT EVERY TIME

Edit → Stage → Commit → Sync

Working folder

1 Edit

Change files (you or an AI) and save.

Staging area

2 Stage

Click **+** to choose what goes in the snapshot.

Local repo

3 Commit

Save the snapshot with a clear message.

GitHub

4 Sync

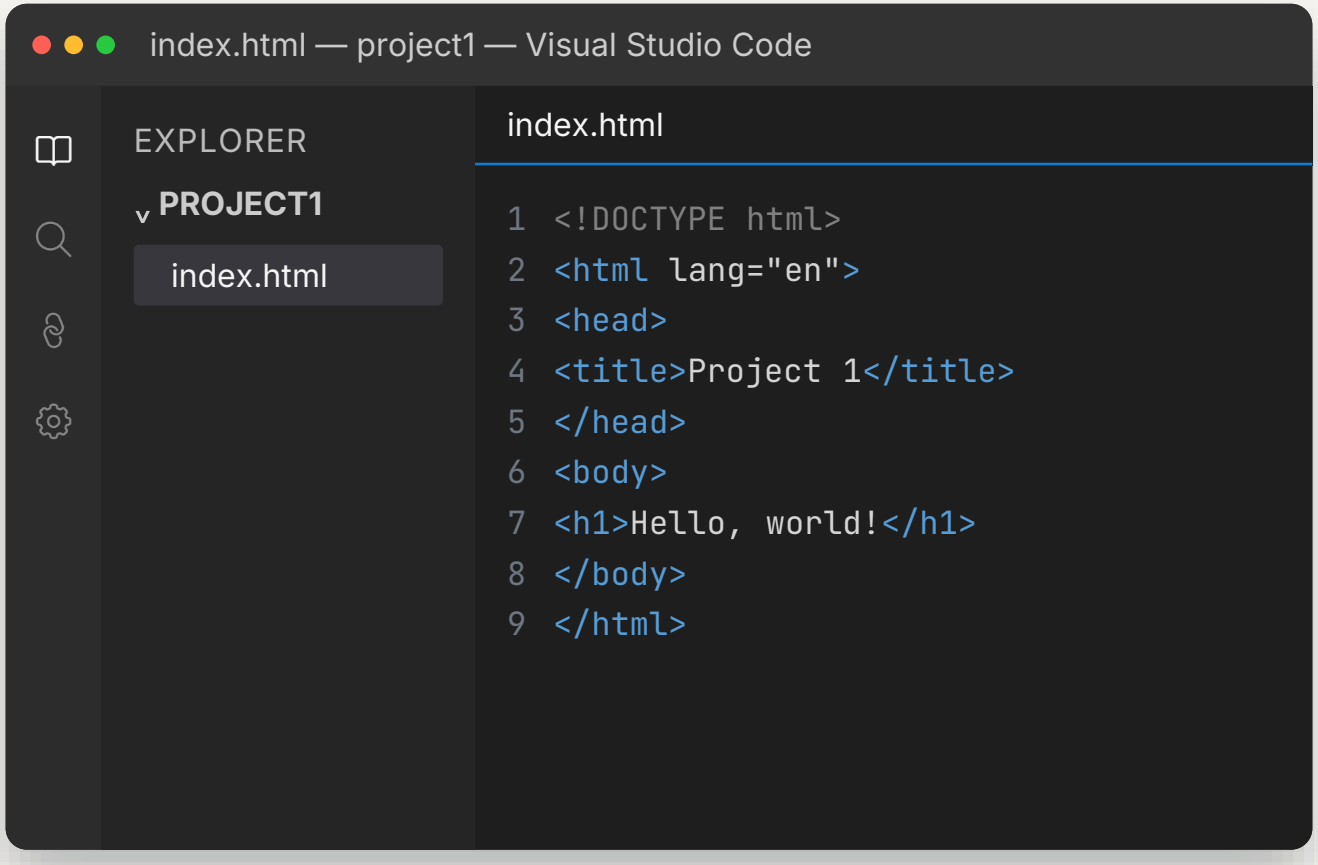
Upload commits so they're backed up online.

A commit is a **checkpoint** : you can always come back to it.

STEP 1 · CREATE THE PROJECT

Make a folder and a home page

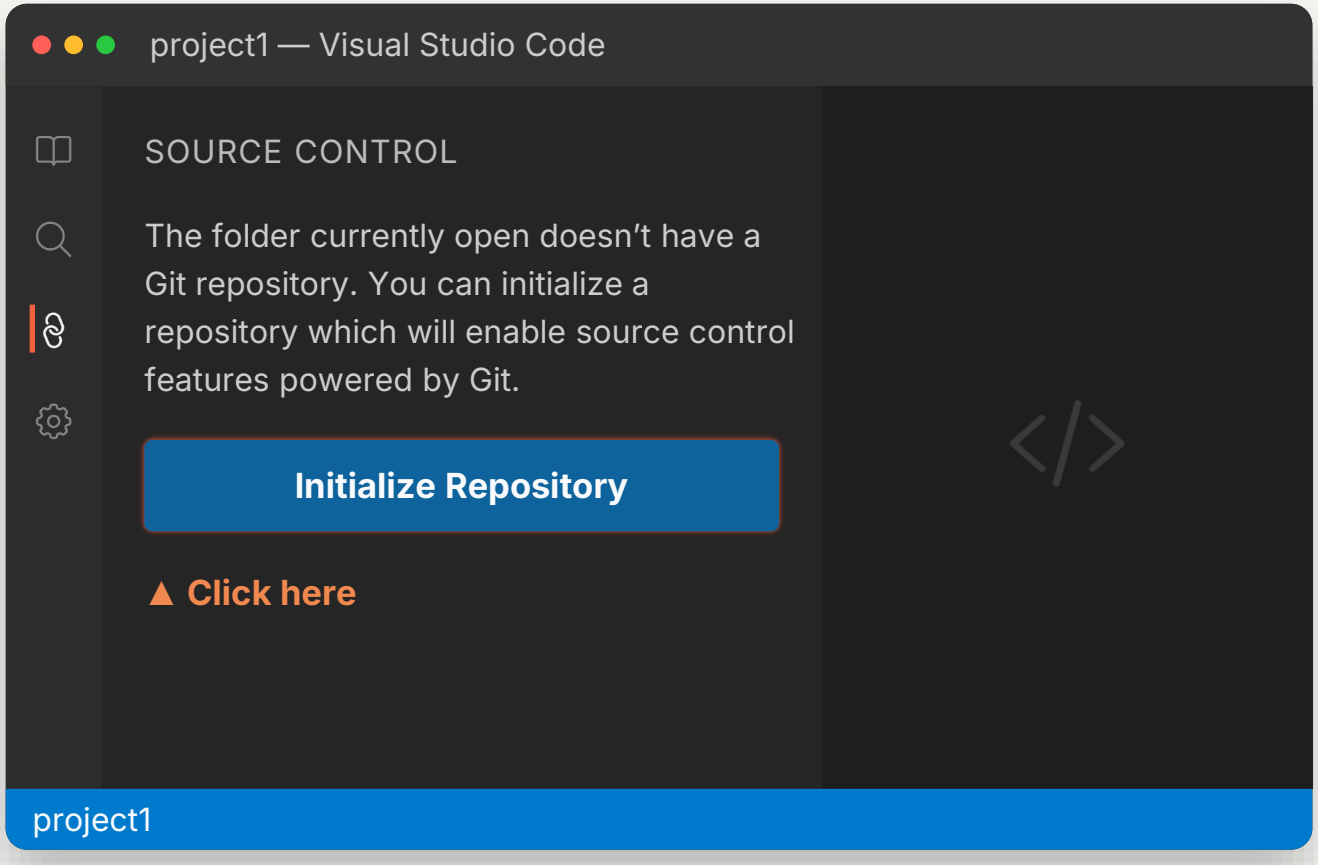
1. Create a folder named **project1** (no spaces).
2. VS Code: **File** ▸ **Open Folder** ▸ choose project1.
3. New File ▸ name it **index.html**. Web hosts look for this exact name.
4. Type **!** then **Tab** for a starter page, or ask your AI assistant.
5. Save with **Ctrl+S** (Cmd+S on Mac).



STEP 2 · INITIALIZE

Turn the folder into a Git repository

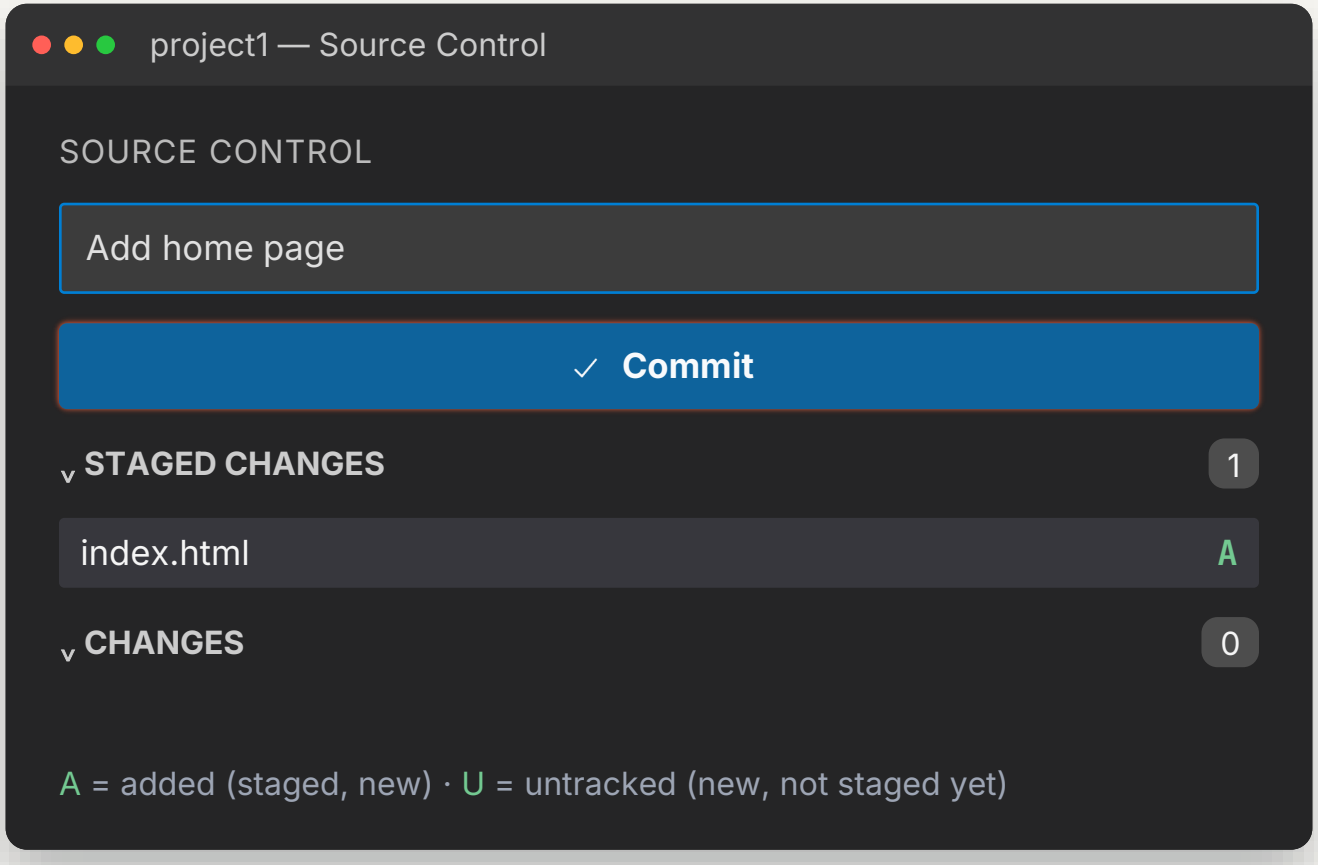
1. Click the **Source Control** icon in the left bar (**Ctrl+Shift+G**).
2. Click **Initialize Repository**.
3. Git creates a hidden **.git** folder inside project1. That folder *is* the repository.
4. Can't see it? File Explorer ▸ **View** ▸ **Show** ▸ **Hidden items**. Never edit or delete it.



STEP 3 · STAGE & COMMIT

Save your first snapshot

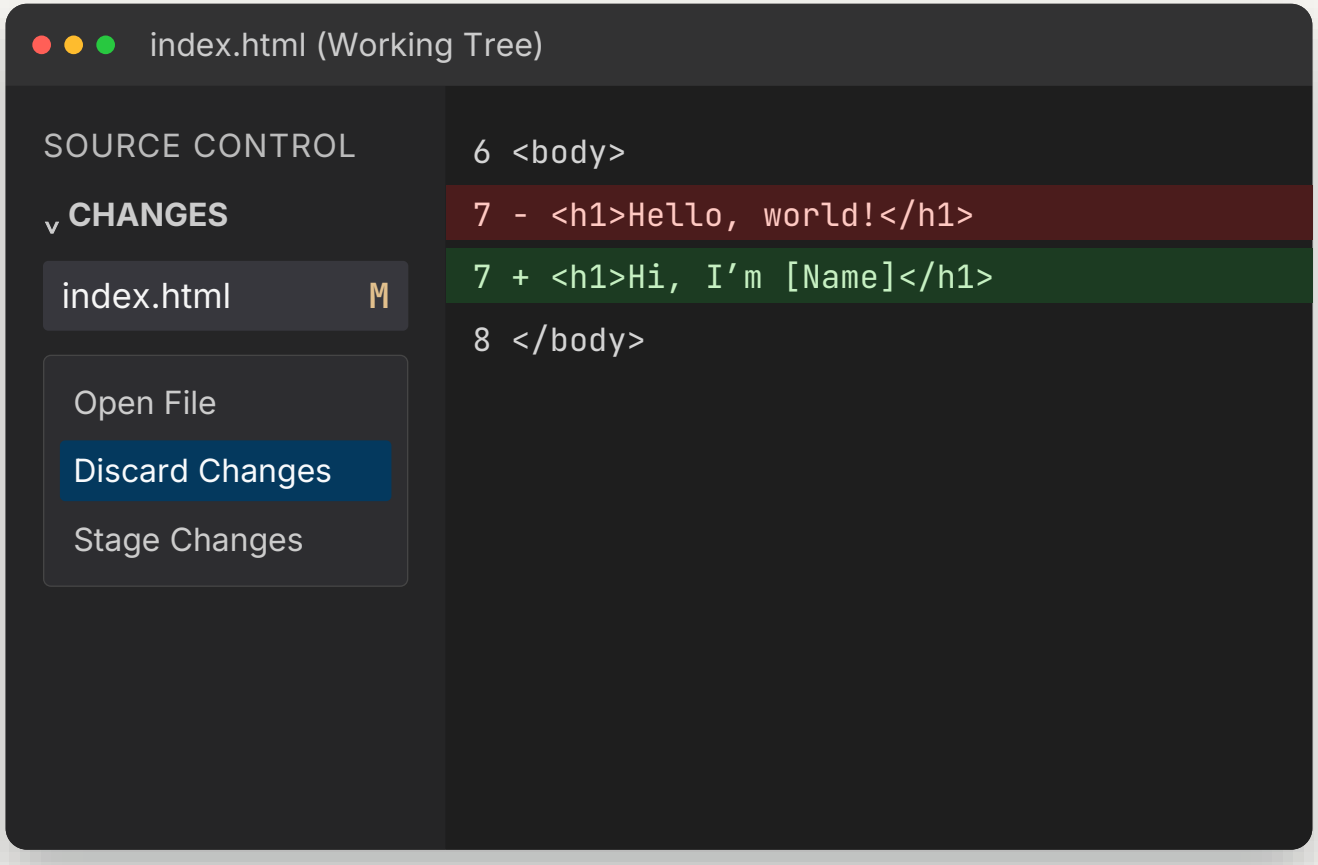
1. Under **Changes**, hover over index.html and click **+**. It moves to **Staged Changes**.
2. Type a message in the box: **Add home page**.
3. Click **Commit**.
4. Good messages say *what* changed: “Add contact form”, not “stuff”.
5. Error about user.name? Go back to **Setup B**.



STEP 4 · SEE & UNDO CHANGES

Git shows every line you change

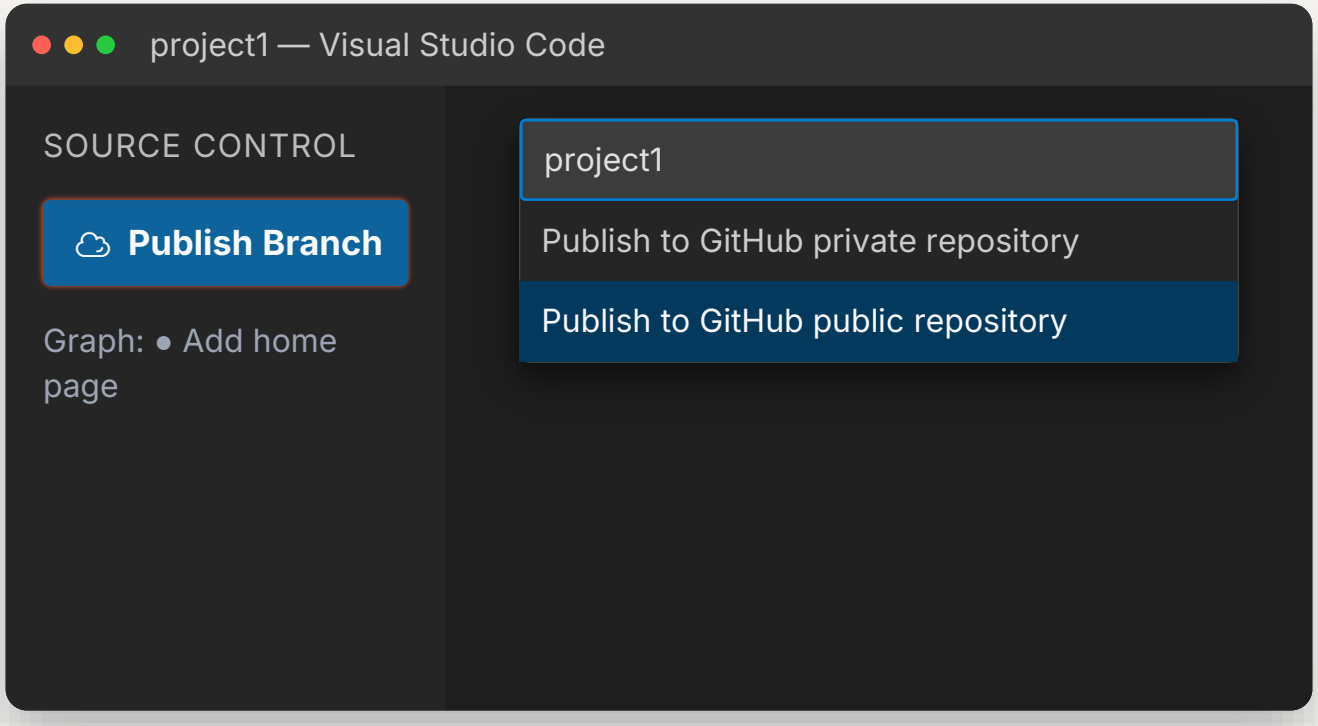
1. Change the h1 text to your name and save.
2. Source Control shows index.html with **M** (modified).
3. Click the file to open the **diff**: **red = removed**, **green = added**.
4. Don’t want it? Right-click ▸ **Discard Changes**. The file returns to the last commit.
5. Careful: a discard cannot be undone.



STEP 5 · PUBLISH TO GITHUB

Put the repository online

1. In Source Control, click **Publish Branch**.
2. Click **Allow**, sign in to GitHub in the browser, then **Open Visual Studio Code**.
3. Choose **private** (only you) or **public** (anyone with the link).
4. Wait for “Successfully published”, then click **Open on GitHub**.

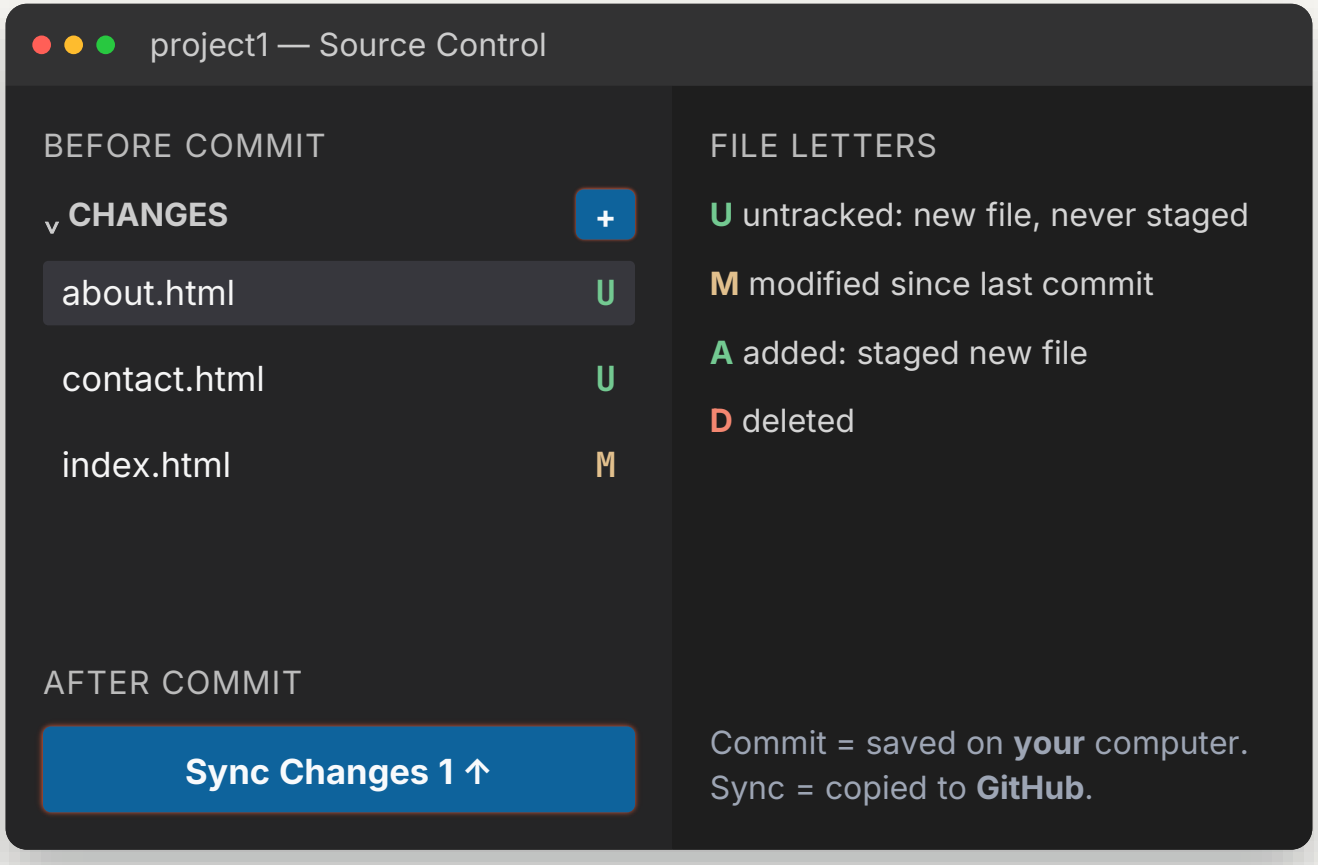


`github.com/username/project1`

STEP 6 · KEEP GOING

Add pages, commit, then Sync

1. Create **about.html** and **contact.html**; link all three pages in a nav menu.
2. Click **+** on the **Changes** header to stage all files.
3. Commit: **Add about and contact pages**.
4. Click **Sync Changes ↑1** to upload.
5. Refresh GitHub: you now see **2 commits**. Click it to view the history.

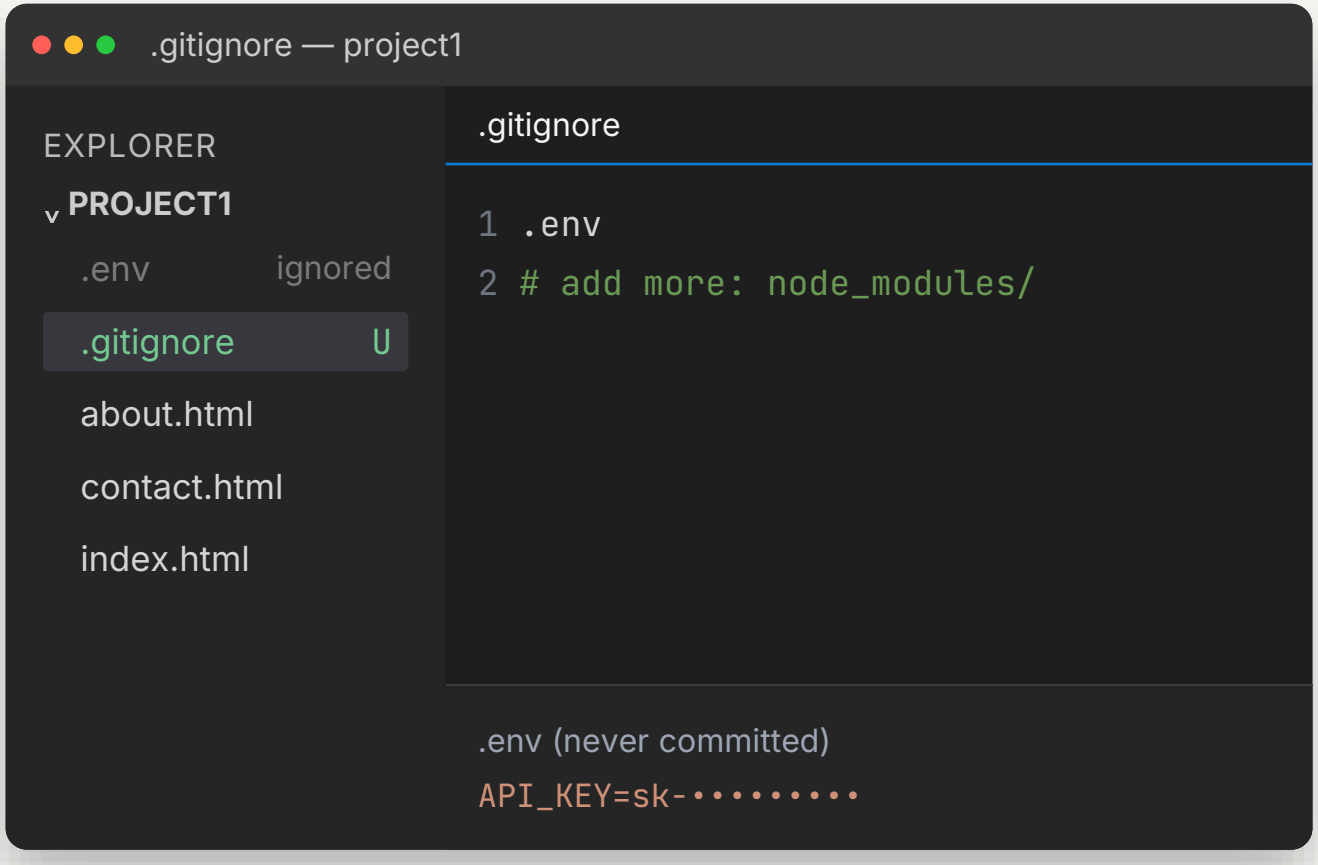


STEP 7 · PROTECT SECRETS

Keep passwords off GitHub with .gitignore

1. Secrets (API keys, passwords) go in a file named **.env**.
2. In Source Control, right-click .env ▸ **Add to .gitignore**.
3. A **.gitignore** file appears listing .env. Commit and sync it.
4. .env now stays on your computer only; it turns grey in Explorer.

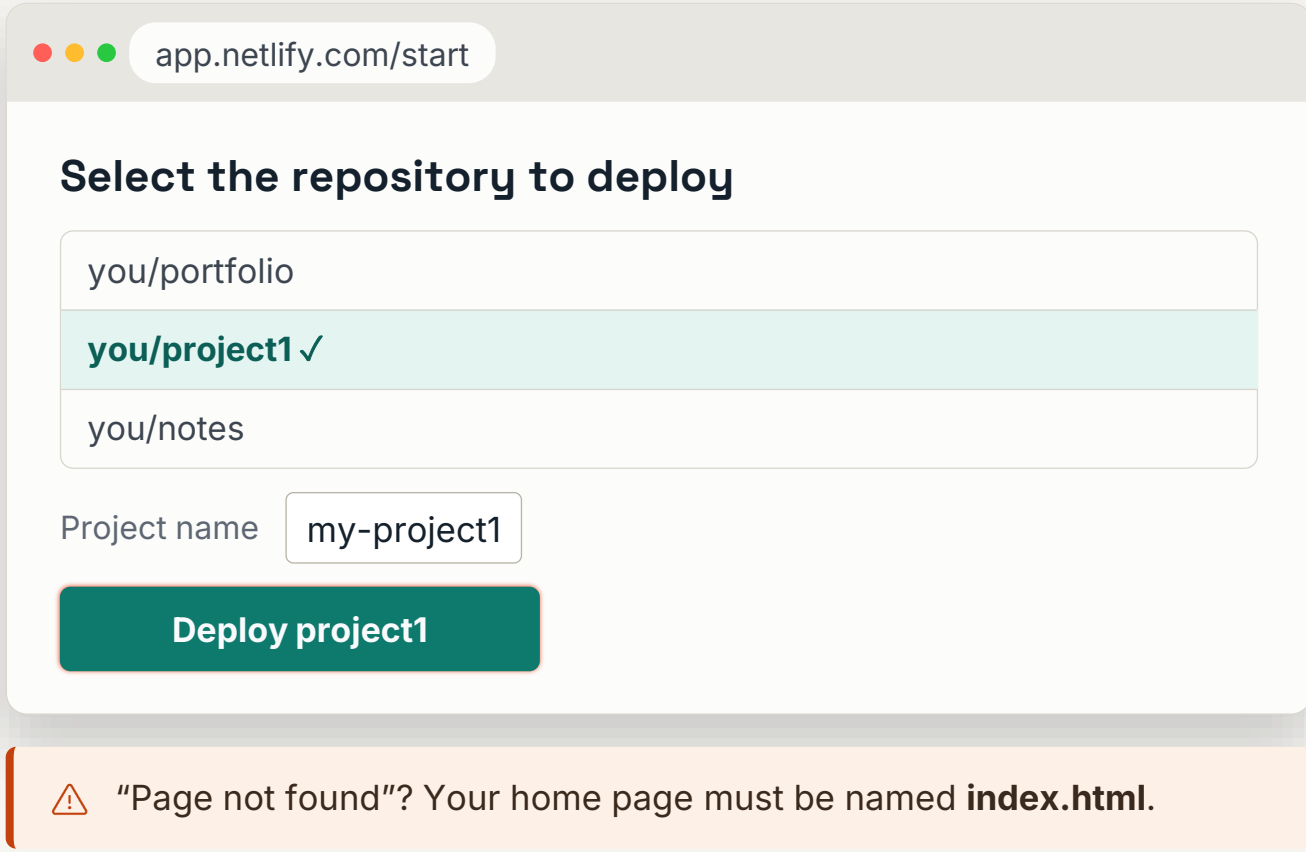
> ⚠ Already pushed a secret? Deleting the file isn't enough; it stays in history. **Revoke the key** and make a new one.



STEP 8 · GO LIVE

Deploy the site free with Netlify

1. Go to **netlify.com** and sign up with GitHub.
2. **Add new project ▸ Import an existing project ▸ GitHub**.
3. Pick **project1**, give it a name, click **Deploy**.
4. Open your **name.netlify.app** link.
5. Every sync to **main** now redeploys the site automatically.



⚠ “Page not found”? Your home page must be named **index.html**.

STEP 9 · WORK AS A TEAM

Clone the project on another computer

1. On GitHub, click the green **Code** button and copy the HTTPS link.
2. On the teammate's computer, open a terminal in any folder.
3. Run **git clone** + the link. A full copy, with history, arrives.
4. Open that folder in VS Code (**code .**).
5. Teammate pushed? Click **Fetch**, then **Sync Changes ↓** to pull their commits.

PowerShell — Teammate's PC

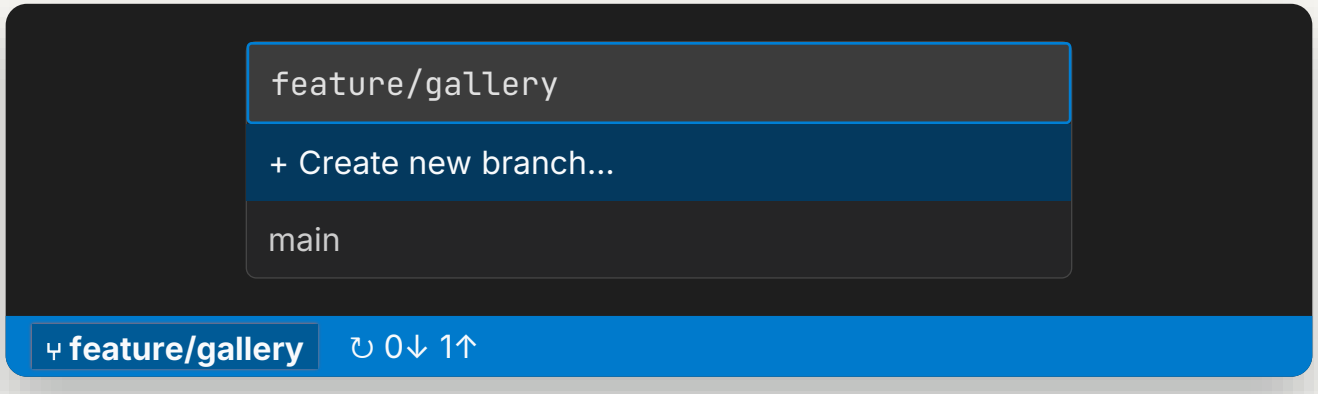
```
PS C:\lab> git clone https://github.com/you/project1.git
Cloning into 'project1'...
Receiving objects: 100%, done.
PS C:\lab> cd project1
PS C:\lab\project1> code .
```

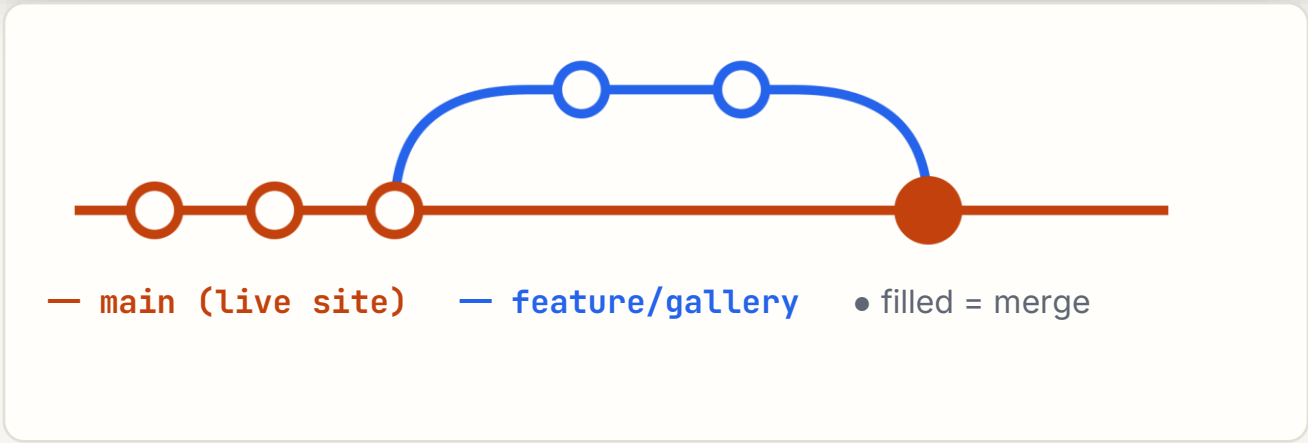


STEP 10 · BRANCHES

Build new features on a branch

1. Click **main** in the bottom-left status bar.
2. Choose **Create new branch…** and name it after the feature: **feature/gallery**.
3. Build gallery.html, stage, commit. Commits land on *this* branch.
4. Click **Publish Branch** to put it on GitHub.
5. Switching branches swaps the files in your folder. Commit first.

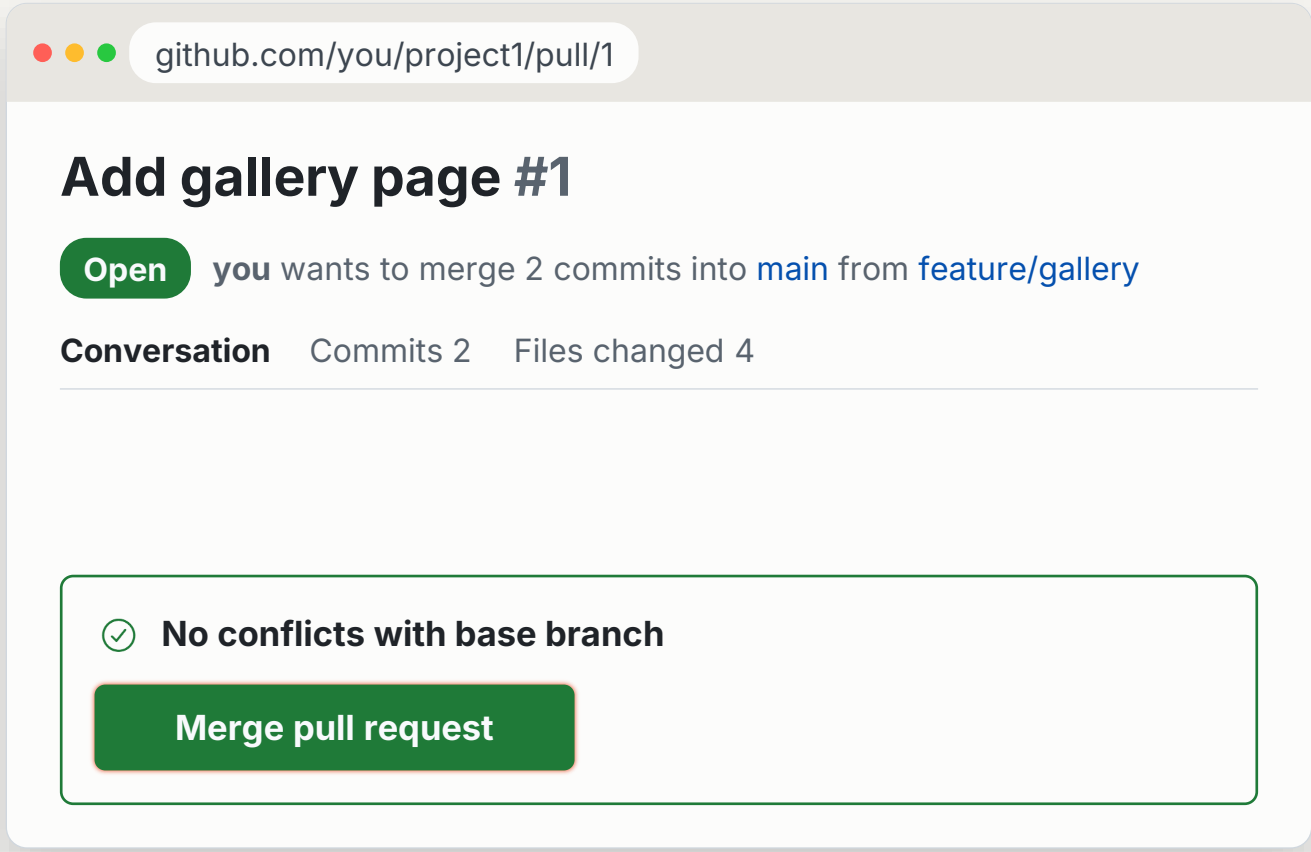




STEP 11 · PULL REQUEST & MERGE

Merge the feature into main

1. On GitHub, click **Compare & pull request**.
2. Add a title and description, then **Create pull request**.
3. Review the **Files changed** tab (a teammate can comment).
4. Click **Merge pull request** ▸ **Confirm merge**. Netlify redeploys.
5. Delete the branch. In VS Code, switch to **main** and **Sync**.



Git Revision

Every VS Code button is a Git command

VS Code action	Git command	What it does
Initialize Repository	git init	Start tracking this folder
+ (Stage)	git add <file>	Pick changes for the next snapshot
Commit	git commit -m "message"	Save a snapshot
Publish Branch / Sync ↑	git push	Upload commits to GitHub
Fetch, then Sync ↓	git pull	Download teammates' commits
Discard Changes	git restore <file>	Undo edits since the last commit
Create new branch	git switch -c <name>	Start a feature branch
Click branch name	git switch <name>	Move to another branch
(terminal)	git clone <url>	Copy a repository to a new computer
Source Control panel	git status / git log	See what changed / the history

Lab Time